

Міністерство освіти та науки України

Національний університет “Львівська політехніка”

Інститут інформаційно-комунікаційних технологій та електронної
інженерії



ЗВІТ

З Лабораторної роботи 1
з дисципліни “Об’єктно-орієнтовне програмування частина 2”

Виконав:

Ст. гр. ІХ-22

Ненадовський Р.М.

Прийняв:

Плесканка М.В.

Львів 2026

Тема: Застосування принципів SOLID об'єктно-орієнтованого програмування при проєктуванні програмних систем у сфері телекомунікацій.

Мета: Ознайомитися з принципами SOLID, навчитися застосовувати їх під час проєктування класів на Python та формувати навички створення гнучкого й підтримуваного коду.

Завдання:

1. Реалізація принципу SRP

Необхідно перепроєктувати класи так, щоб кожен клас мав лише одну відповідальність:

- розділити формування звіту і його збереження;
- розділити логіку абонента (дані, SMS, баланс).

2. Реалізація принципу OCP

Розробити систему тарифів:

- створити базовий клас тарифу;
- додати нові тарифи через наслідування без зміни існуючого коду.

3. Реалізація принципу LSP

Забезпечити коректну роботу підкласів:

- підкласи повинні повністю замінювати базовий клас без зміни поведінки програми.

4. Реалізація принципу ISP

Розділити інтерфейси:

- створити кілька спеціалізованих інтерфейсів замість одного універсального.

5. Реалізація принципу DIP

Забезпечити інверсію залежностей:

- класи повинні залежати від абстракцій, а не від конкретних реалізацій.

Код:

```
# Завдання 1 - SRP
class Subscriber:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

class SMSService:
    def send_sms(self, subscriber, message):
        print(f"SMS для {subscriber.name}: {message}")

class BalanceCalculator:
    def calculate(self, subscriber):
        print(f"Баланс абонента {subscriber.name}: {subscriber.balance}")
```

```
# Завдання 2 - OCP
class Tariff:
    def calculate_price(self, minutes):
        pass
```

```
class BasicTariff(Tariff):
    def calculate_price(self, minutes):
        return minutes * 1
```

```
class PremiumTariff(Tariff):
    def calculate_price(self, minutes):
        return minutes * 0.5
```

```
# Завдання 3 - LSP
class Bird:
    def move(self):
        print("Птах рухається")
```

```
class FlyingBird(Bird):
    def move(self):
        print("Птах літає")
```

```
class WalkingBird(Bird):
    def move(self):
        print("Птах ходить")
```

```
# Завдання 4 - ISP
class Printer:
    def print_document(self, text):
        print("Друк:", text)
```

```
class Scanner:
    def scan_document(self):
        print("Сканування документа")
```

```
# Завдання 5 - DIP
class Database:
    def save(self, data):
        print("Збережено в базу:", data)
```

```
class UserService:
    def __init__(self, database):
        self.database = database
```

```
    def create_user(self, name):
        self.database.save(name)
```

```

# Запуск програми
print("\n--- Завдання 1 (SRP) ---")

subscriber = Subscriber("Ivan", 100)

sms = SMSService()
sms.send_sms(subscriber, "Привіт!")

balance = BalanceCalculator()
balance.calculate(subscriber)

print("\n--- Завдання 2 (OCP) ---")

basic = BasicTariff()
premium = PremiumTariff()
print("Basic тариф:", basic.calculate_price(10))
print("Premium тариф:", premium.calculate_price(10))

print("\n--- Завдання 3 (LSP) ---")
bird1 = FlyingBird()
bird2 = WalkingBird()

bird1.move()
bird2.move()

print("\n--- Завдання 4 (ISP) ---")
printer = Printer()
scanner = Scanner()

printer.print_document("Лабораторна робота")
scanner.scan_document()

print("\n--- Завдання 5 (DIP) ---")
db = Database()
user_service = UserService(db)
user_service.create_user("Petro")

```

Результат:

```

--- Завдання 1 (SRP) ---
SMS для Ivan: Привіт!
Баланс абонента Ivan: 100

--- Завдання 2 (OCP) ---
Basic тариф: 10
Premium тариф: 5.0

--- Завдання 3 (LSP) ---
Птах літає
Птах ходить

--- Завдання 4 (ISP) ---
Друк: Лабораторна робота
Сканування документа

--- Завдання 5 (DIP) ---
Збережено в базу: Petro

Process finished with exit code 0

```

Висновок: Під час роботи було вивчено п'ять основних принципів SOLID (SRP, OCP, LSP, ISP, DIP), що дозволяють створювати масштабовану архітектуру програмного забезпечення, уникати надмірної зв'язності компонентів та полегшувати подальше тестування й супровід коду.